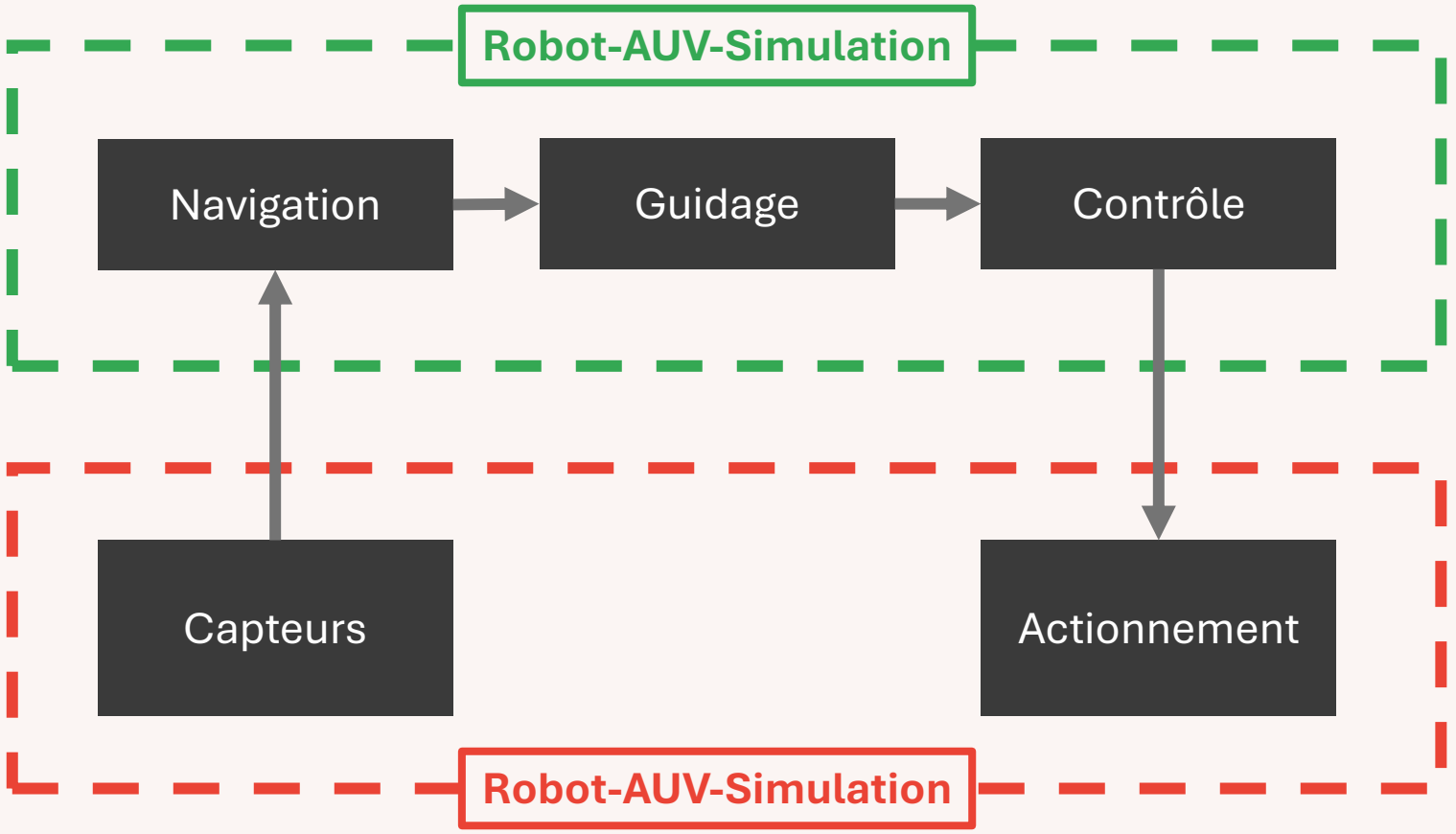
CRÉER UNE ARCHITECTURE ROS2

En quelques étapes pour ROS2 Python

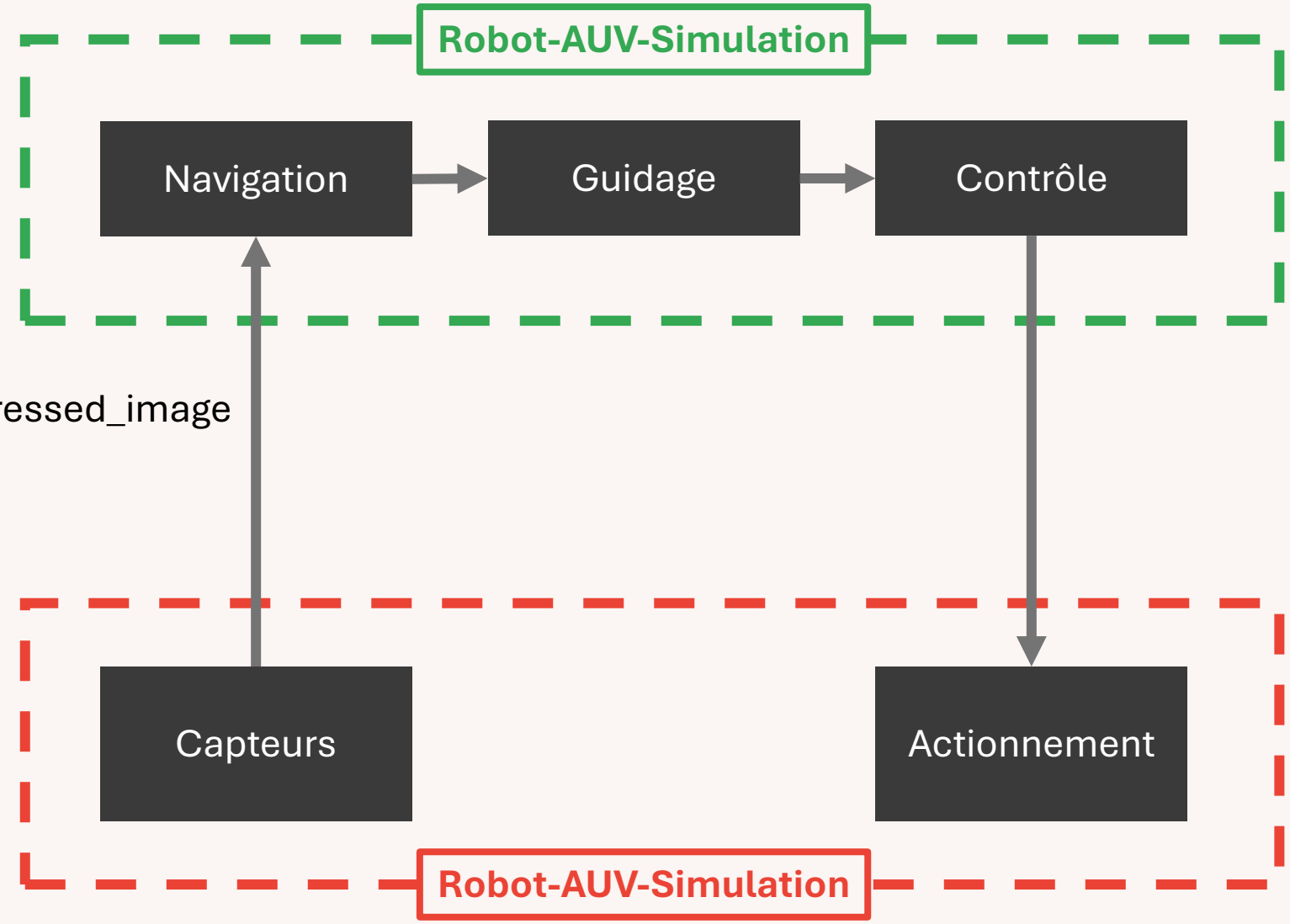
ENSTA | INSTITUT POLYTECHNIQUE DE PARIS



Architecture logicielle classique



Architecture logicielle classique



Etape 0 : sourcer votre environnement

Première utilisation ?
Pour télécharger ROS2 : https://youtu.be/ikKELwBJ-1U

Ouvrez un terminal et sourcez votre environnement

```
source /opt/ros/humble/setup.bash
```

→ Vous pouvez mettre cette ligne à la fin de votre bashrc pour que la commande se lance à chaque ouverture du terminal.

Etape 1 : créer un workspace et package

Créer un workspace ROS2 et s'y déplacer :

mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src

Créer un nouveau package ROS2 python

ros2 pkg create --build-type ament_python nom_du_package

→ Cela crée un dossier nom_du_package/ avec la structure suivante :

nom_du_package /
- package.xml
- setup.cfg
- setup.py
- nom_du_package/
  - __init__.py
- resource/
  - nom_du_package

Étape 3 : Ajouter un script exécutable

Dans nom_du_package /nom_du_package/, créez votre premier nœud

nano simuROV/nom_du_package/simuROV/main.py

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class SimpleNode(Node):
    def __init__(self):
        super().__init__('simple_node')
        self.get_logger().info("Hello from simuROV!")

def main(args=None):
    rclpy.init(args=args)
    node = SimpleNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Classique pour tous vos nodes python

Classique pour tous vos nodes python

nom_du_package /

- package.xml
- setup.cfg
- setup.py
- nom_du_package/
  - __init__.py
  - main.py
- resource/
  - nom_du_package

Étape 4 : Modifier setup.py

Ouvrez setup.py et ajoutez votre script dans la section entry_points :

```
from setuptools import setup

package_name = ‘nom_du_package’

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ton_nom',
    maintainer_email='ton_mail@exemple.com',
    description='Simulation d’un ROV sous ROS 2',
    license='Apache License 2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            ‘nom_du_node = nom_du_package.nom_du_code:main’,
        ],
    },
)
```

nom_du_package /
- package.xml
- setup.cfg
- setup.py
- nom_du_package/
  - __init__.py
  - main.py
- resource/
  - nom_du_package

À changer pour le bon fonctionnement du package

À changer pour avoir un package propre

Étape 5 : Construire le package

⚠ **Depuis le workspace :**

cd ~/ros2_ws
colcon build --packages-select nom_du_package

Puis sourcez votre environnement :

source install/setup.bash

Étape 6 : Lancer votre nœud

Lancez votre nœud seul :

ros2 run nom_du_package main

Lancer un fichier launch :

ros2 launch nom_du_package nom_du_fichier.launch.py

Commandes annexes :

Lister les topics disponibles

ros2 topic list

Afficher les messages d’un topic

ros2 topic echo /nom_du_topic

Afficher les informations d’un nœud

ros2 node info /nom_du_noeud

Merci pour votre attention

ENSTA | INSTITUT POLYTECHNIQUE DE PARIS